

STEP 1: Install & Import Libraries

```
# Install required libraries
!pip install nltk scikit-learn pandas numpy
```

```
import nltk
import numpy as np
import pandas as pd
import string
```

```
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

STEP 2: Download NLTK Data

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

STEP 3: Create Dataset (15 Documents)

```
documents = [
    "Machine learning is a part of artificial intelligence.",
    "AI includes machine learning and deep learning.",
    "Deep learning is a subset of machine learning.",
    "Natural language processing deals with text data.",
    "NLP is used for text analysis and understanding.",

    # Modified / similar
    "Machine learning belongs to artificial intelligence.",
    "Artificial intelligence includes machine learning.",

    # Paraphrased
    "Computers learn from data using machine learning.",
    "Text processing is done using NLP techniques.",

    # Different
    "The sky is blue and beautiful.",
    "I love playing cricket in the evening.",
    "Dogs are loyal animals.",
    "Cooking is an important life skill.",
    "Music makes people happy.",
    "Traveling is fun and exciting."
]

df = pd.DataFrame(documents, columns=["Text"])
df.head()
```

	Text 
0	Machine learning is a part of artificial intel...
1	AI includes machine learning and deep learning.
2	Deep learning is a subset of machine learning.
3	Natural language processing deals with text data.
4	NLP is used for text analysis and understanding.

Next steps: [Generate code with df](#) [New interactive sheet](#)

STEP 4: Preprocessing

```
nlTK.download('punkt_tab', quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    # Lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Tokenization
    tokens = word_tokenize(text)

    # Stopword removal
    tokens = [word for word in tokens if word not in stop_words]

    # Lemmatization
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return " ".join(tokens)

df["Processed"] = df["Text"].apply(preprocess)
df.head()
```

STEP 5: TF-IDF (for Cosine Similarity)

```
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(df["Processed"])
```

STEP 6: Cosine Similarity

```
cos_sim = cosine_similarity(tfidf_matrix)

cos_df = pd.DataFrame(cos_sim)
cos_df.head()
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	1.000000	0.299827	0.330438	0.000000	0.000000	0.655732	0.685397	0.201200	0.000000	0.0	0.0	0.0	0.0	0.0	0.0
1	0.299827	1.000000	0.649102	0.000000	0.000000	0.299827	0.537241	0.248996	0.000000	0.0	0.0	0.0	0.0	0.0	0.0
2	0.330438	0.649102	1.000000	0.000000	0.000000	0.330438	0.345387	0.274417	0.000000	0.0	0.0	0.0	0.0	0.0	0.0
3	0.000000	0.000000	0.000000	1.000000	0.127318	0.000000	0.000000	0.162553	0.271769	0.0	0.0	0.0	0.0	0.0	0.0
4	0.000000	0.000000	0.000000	0.127318	1.000000	0.000000	0.000000	0.000000	0.294359	0.0	0.0	0.0	0.0	0.0	0.0

Next steps: [Generate code with cos_df](#) [New interactive sheet](#)

```
pairs = []

n = len(cos_sim)

for i in range(n):
    for j in range(i+1, n):
        pairs.append((i, j, cos_sim[i][j]))

# Sort by similarity score (descending)
pairs = sorted(pairs, key=lambda x: x[2], reverse=True)

print("Top 5 Similar Pairs (Cosine):\n")

for i, j, score in pairs[:5]:
    print(f"Doc {i} - Doc {j} : {score:.3f}")
```

Top 5 Similar Pairs (Cosine):

```
Doc 0 - Doc 6 : 0.685
Doc 5 - Doc 6 : 0.685
Doc 0 - Doc 5 : 0.656
Doc 1 - Doc 2 : 0.649
Doc 1 - Doc 6 : 0.537
```

STEP 7: Bag of Words

```
def jaccard_similarity(doc1, doc2):
    set1 = set(doc1.split())
    set2 = set(doc2.split())
    return len(set1 & set2) / len(set1 | set2)
```

```
jaccard_scores = []

for i in range(len(df)):
    for j in range(i+1, len(df)):
        score = jaccard_similarity(df["Processed"][i], df["Processed"][j])
        jaccard_scores.append((i, j, score))

jaccard_scores = sorted(jaccard_scores, key=lambda x: x[2], reverse=True)

print("Top 5 Similar Pairs (Jaccard):\n")
for i, j, score in jaccard_scores[:5]:
    print(f"Doc {i} - Doc {j} : {score:.3f}")
```

Top 5 Similar Pairs (Jaccard):

```
Doc 0 - Doc 5 : 0.667
Doc 0 - Doc 6 : 0.667
Doc 5 - Doc 6 : 0.667
Doc 1 - Doc 2 : 0.500
Doc 1 - Doc 6 : 0.429
```

STEP 8: WordNet Semantic Similarity

```
from nltk.corpus import wordnet as wn

def wordnet_similarity(word1, word2):
    syn1 = wn.synsets(word1)
    syn2 = wn.synsets(word2)

    if syn1 and syn2:
        return syn1[0].wup_similarity(syn2[0])
    return 0
```

```
sentence_pairs = [
    ("machine learning", "artificial intelligence"),
    ("text processing", "nlp"),
    ("dog", "animal"),
    ("car", "vehicle"),
    ("music", "song"),
    ("sky", "blue"),
    ("happy", "joyful"),
    ("run", "walk"),
    ("eat", "consume"),
    ("big", "large")
]

for w1, w2 in sentence_pairs:
    print(f"{w1} - {w2} :", wordnet_similarity(w1, w2))
```

```
machine learning - artificial intelligence : 0
text processing - nlp : 0
dog - animal : 0.875
car - vehicle : 0.8
```